

Project 1: RAG and Context Engineering — Project Report

1. Data Creation Methodology

We created a golden evaluation set of 20 hand-labeled Q&A pairs over the RapidFire AI OSS documentation corpus. The goal was to test whether a RAG system could retrieve the right documentation fragments and synthesize technically correct answers under a fixed context budget. Each question was written against one or more .rst source files and paired with explicit source evidence spans (file name and line range). Questions were iteratively refined to remove those guessable without retrieval, too broad, or with evidence spans too diffuse to be grounding. LLMs were used to assist drafting; all pairs were proofread, answer-checked against the corpus, and updated with precise file+line evidence before inclusion.

The final set covers five question types, distributed as shown in Table 1:

Table 1: Golden Q&A Pair Type Breakdown (20 total). Actual distribution from generated-golden-qa-pairs.json; each entry carries a question_type field.

Question Type	Count	Example Question
Factual	4	"What are the three CI strategies supported by online aggregation?"
Conceptual	6	"Why does the adaptive engine reduce config comparison time?"
Procedural	2	"What workflow is required to set up RAG eval on a CPU-only machine?"
Comparative	6	"What tradeoffs exist between FAISS, Pinecone, and PGVector?"
Multi-hop	2	"How do GSM8K, FiQA, and SciFact tutorials differ in embedding/model choices?"

Ambiguity was reduced by scoping each question to information explicitly present in the corpus and writing reference answers at the level of detail supported by the evidence spans — neither merely naming a concept nor adding unsupported claims. For multi-concept questions, multiple source evidence entries were included rather than expanding one span into noisy territory. The set balances retrieval difficulty across direct, neighboring, and multi-document evidence.

Multi-hop questions (IDs 13, 16) require evidence from two or three source files; these were especially useful for stressing the retriever's recall because any single perfectly-ranked file still produces an incomplete answer. The comparative questions (IDs 2, 3, 8, 10, 12, 15) probe the system's ability to surface parallel sections from different parts of the documentation, which char n-gram retrieval handled well given the precision of API names.

Custom eval metric explored: We considered a Source Precision metric that checks whether the generated answer cites only files actually present in the retrieved context — catching hallucinated source attribution where the model references a file it was not given. While not added to the final judge pipeline (the standard faithfulness metric partially captures this), it informed manual inspection of the 4 faithfulness failures, where the model drew on implicit knowledge rather than the supplied chunks.

2. Final Pipeline Architecture

The submitted pipeline (main.py) accepts an input JSON file (question_id, question), retrieves evidence from the documentation corpus in data/sourcedocs, calls a Triton-hosted LLM, and writes the required output fields: question_id, answer, retrieved_context, and sources. Figure 1 shows the end-to-end architecture.

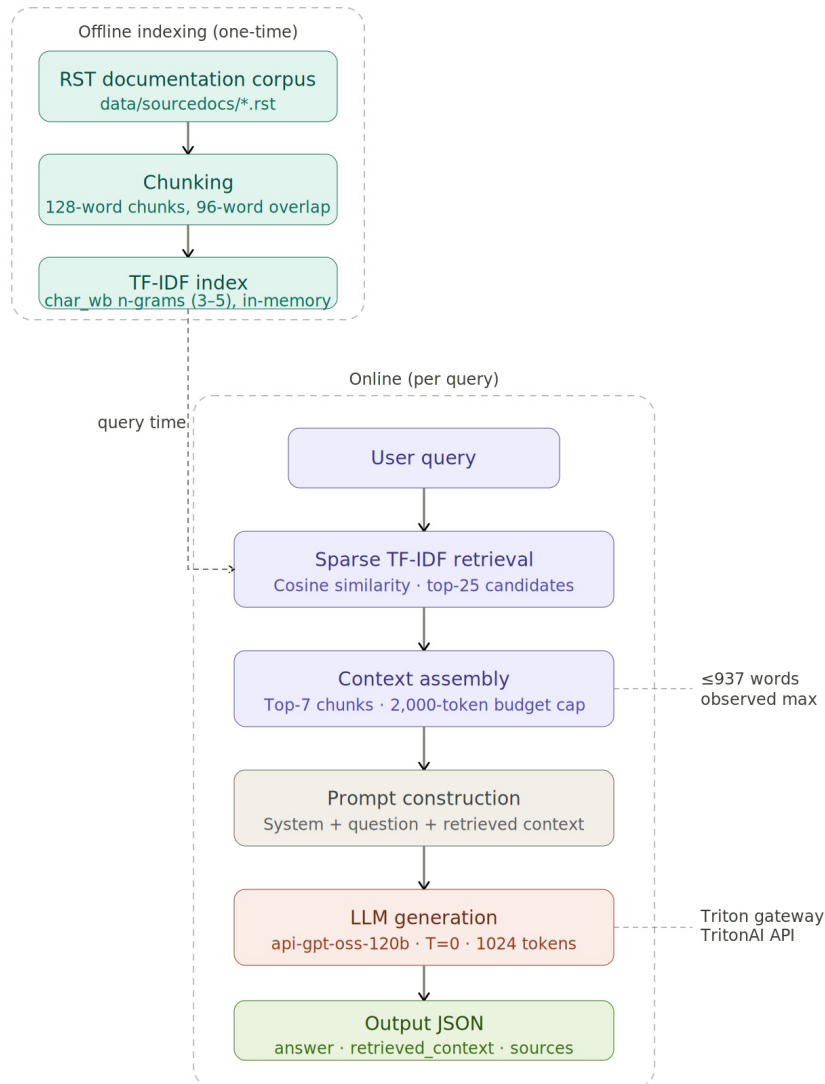


Figure 1: End-to-end RAG pipeline. The offline path (top-left, dashed) runs once: RST corpus → 128-word/96-overlap chunker → TF-IDF char_wb index. The online path (right, dashed) runs per query: user query → sparse retrieval (top-25 candidates) → context assembly with 2,000-token cap (≤ 7 chunks, ≤ 937 words observed) → prompt construction → api-gpt-oss-120b via Triton ($T=0$, 1024 tok) → output JSON with answer, retrieved_context, sources.

Retriever

The final retriever uses a sparse TF-IDF representation over character n-grams (char_wb, length 3–5) rather than a dense embedding model. This worked well because the corpus is rich in API identifiers and configuration keys (e.g., RGridSearch, RLangChainRagSpec, enable_gpu_search). Dense embeddings smooth over lexical details; character n-grams preserve partial matches and tolerate slight wording differences between question and documentation. The vector store is an in-memory cosine-similarity matrix — fast, deterministic, and suited to a small but lexically precise corpus. No FAISS or PGVector backend was used in the final pipeline.

Chunking Strategy

128-word chunks with 96-word overlap (75% overlap ratio). Small chunks keep each retrieved unit focused; large overlap prevents important API descriptions from being split across boundaries. The retriever considers the top-25 candidates and serializes up to 7 context chunks, deduplicating source files where possible while preserving

high-scoring repeated evidence.

Context Assembly and Budget Enforcement

Context chunks are serialized with source-file prefixes and clear delimiters (---). Token count is approximated by word count (conservative, no tokenizer dependency). Chunks are added in ranked order until the 2,000-token budget would be exceeded. A single-chunk edge case is handled: if one chunk alone exceeds the budget, it is word-truncated rather than skipped. Maximum observed retrieved context in the validation run was 937 words, leaving comfortable margin for the system prompt and question. Known limitation: source line spans in sources output are whole-file approximations (e.g., lines [1, 134]); the pipeline correctly identifies the source file but does not recover chunk-level line offsets at query time.

Prompt Design and Generation

Generator: `api-gpt-oss-120b` via Triton's OpenAI-compatible API. Temperature: 0. `max_completion_tokens`: 1024. The system prompt (`reference_like` variant, selected after comparing two prompt strategies — see Section 3) instructs the model to answer only from retrieved evidence, use precise documentation wording and include key parameter names and defaults, and state explicitly when evidence is insufficient. No cross-encoder reranker was applied: early experiments showed it added latency without reliably improving judged scores, as the char n-gram retriever already achieved high recall and remaining failures were in answer synthesis rather than candidate ordering.

3. Experimentation Methodology

We used RapidFire AI's multi-config framework with `RFGridSearch` over an explicit List of 8 config dictionaries, run inside a single `Experiment(mode="evals")` call named `discussion1-rag-docs-tuning-8configs`. `RFGridSearch` was appropriate because the search space was small and we wanted to isolate individual design choices cleanly — it enumerates all combinations deterministically, unlike `RFRandomSearch`, which is preferred for larger continuous spaces. The config set was defined as a `List(...)` of 8 explicit dictionaries passed to `RFGridSearch`, each varying retrieval and chunking knobs while holding the dataset, evaluation metrics, and online aggregation strategy constant.

Each of the 8 runs processed the same 45-question validation set with the same generation model, keeping comparisons focused on retrieval and chunking knobs: chunk size, overlap, analyzer type (word 1–2 grams vs. char_wb 3–5 grams), and `max_completion_tokens`. Before the main sweep, two prompt strategies — `concise` and `reference_like` — were compared on a subset using `generate_and_judge_configs.py`. The `reference_like` prompt, which instructs the model to include precise documentation wording and key parameter names, produced consistently higher faithfulness and completeness and was fixed for the main sweep. This preliminary step was run outside RapidFire to avoid API costs on a full-sweep × 2-prompt grid.

After the sweep, per-run metrics were extracted via a custom script (`extract_all_config_results.py`) that queries RapidFire's MLflow SQLite database directly, collecting params, metrics, and tags for each run into a single JSON. RapidFire's Interactive Control Operations (Stop/Resume) were used mid-run: after early shard results from runs 1–2 (word-based configs) showed a clear retrieval gap relative to runs 3 and 7 (char n-gram configs), the remaining word-based queue was deprioritized in favour of completing char n-gram variants first. This avoided wasting completion budget on configurations that were already demonstrably weaker.

The evaluation used two layers: a fast lexical generation overlap score as a proxy during the RapidFire sweep (cheap and stable for 8-config comparisons), and a slower LLM judge (`claude-sonnet-4-6` via Triton) run only on the best config to obtain correctness, faithfulness, and completeness scores. This separation kept API costs manageable while still producing credible final generation metrics.

Table 2: All 8 RapidFire AI experiment configurations. Run 3* (green) is the final selected pipeline. Gen. Overlap = fast lexical proxy used during the sweep; LLM-judge scores for Run 3* are in Table 3. Total = $0.5 \times \text{Retrieval Score} + 0.5 \times \text{Gen. Overlap}$.

Run	Config Name	Analyzer	Chunk/ Overlap	Max Tok.	Retr. Score	Gen. Overlap	MRR	Total
1	word_192_16	word 1-2g	192/16	1024	0.589	0.654	0.930	0.621
2	word_128_32	word 1-2g	128/32	1024	0.582	0.641	0.907	0.611
3*	char_128_96	char_wb 3-5g	128/96	1024	0.696	0.627	0.848	0.662
4	word_192_16	word 1-2g	192/16	1536	0.589	0.660	0.930	0.624
5	word_96_64	word 1-2g	96/64	1024	0.661	0.625	0.881	0.643
6	word_96_32	word 1-3g	96/32	1024	0.582	0.615	0.896	0.599
7	char_96_64	char_wb 3-5g	96/64	1024	0.662	0.623	0.856	0.642
8	word_192_64	word 1-2g	192/64	1024	0.612	0.655	0.878	0.634

4. Results and Analysis

The final selected configuration (char_128_96, 1024 completion tokens) achieved the best retrieval score and best final judged score across all 8 configs. Full metrics from the LLM judge on the 45-question validation set are shown in Table 3.

Table 3: Best config (char_128_96) — full metric breakdown from LLM judge on 45 validation questions.

Metric	Score	Notes
Precision@5	0.502	Top-ranked file not always the sole relevant file
Recall@5	0.956	Relevant evidence almost always present in top 5
F1@5	0.631	Harmonic mean of P@5 and R@5
Retrieval Score	0.696	Mean of Precision, Recall, F1 @ 5
Correctness (pass rate)	0.867	LLM-judged binary; 6 failures across 45 questions
Faithfulness (pass rate)	0.911	All claims grounded in context; 4 failures
Completeness (0–5)	4.13 / 5	Normalized: 0.827; covers key aspects of reference
Partial Gen. Score	0.868	Mean of 3 normalized generation metrics
Partial Total Score	0.782	$0.5 \times \text{Retrieval} + 0.5 \times \text{Generation}$

Key Findings

Char n-grams beat word tokenization. Runs 3 and 7 (char_wb) achieved the two highest retrieval scores (0.696, 0.662) vs. a maximum of 0.612 for any word-based config. Technical identifiers like RFLangChainRagSpec are brittle as single word tokens but produce rich substring matches under char n-grams, improving recall especially for API and configuration questions.

Recall mattered more than MRR. Run 3 had the lowest MRR (0.848) of any config yet the highest retrieval and total score. For multi-hop questions requiring evidence from multiple files, high recall is decisive — a perfectly-ranked single file still misses co-located evidence. High MRR (run 1: 0.930) did not translate to high total

score.

Overlap ratio is the key chunking lever. Runs 3 (75% overlap) and 7 (67%) both dominated word-based configs. Run 5 (word, 67% overlap) also scored well (0.661), confirming aggressive overlap helps even without char n-grams — but the combination is strongest.

Longer completion tokens did not help. Run 4 (1536 tokens, same retrieval as run 1) improved lexical overlap by only 0.006. When evidence is absent or diluted, longer outputs increase hallucination risk rather than correctness.

Lexical overlap does not predict judged quality. Run 8 scored higher on generation overlap (0.655) than run 3 (0.627), yet run 3 dominated on correctness and faithfulness. Lexical overlap rewards surface similarity; judged metrics penalize unsupported claims regardless of wording match. This is why we used overlap only as an iteration signal, not a final quality claim.

What Surprised Us

The most surprising result was that the winning config had the lowest MRR of all 8 runs. We initially expected MRR to be a reliable proxy for retrieval quality. Instead, it proved a poor predictor of F1@5 — the metric that drives the leaderboard retrieval score. The explanation is that multi-hop questions require multiple files, so a system that ranks one file perfectly but misses the second is penalized heavily by recall. This taught us to prioritize recall-weighted metrics rather than rank-sensitive ones when the evaluation set contains multi-evidence questions. A secondary surprise: aggressive 96-word overlap on 128-word chunks (75%) did not cause significant context redundancy — deduplicated chunk assembly meant the token budget was rarely wasted on repeated passages.

Failure Analysis

6 correctness failures and 4 faithfulness failures across 45 questions. Three main categories: (1) *Retrieval failures* — conceptual question phrasing did not match concrete API-example language in the docs (Q20, Q34); (2) *Context pollution* — high-scoring chunks from adjacent tutorial pages lacked the exact constraint being asked about (Q24); (3) *Generation failures* — broad answers omitting a specific condition required by the reference even when the correct source was retrieved (Q8, Q28, Q45). These explain the gap between high faithfulness (0.911) and the correctness ceiling (0.867).

Future Directions

(1) *Section-aware chunking* on .rst headings — keeps concept-plus-constraints passages intact without needing aggressive overlap. (2) *Query expansion* for API aliases and class names to improve recall on conceptual questions that use synonym phrasing. (3) A *hybrid sparse+dense retriever* (char n-grams + sentence-transformer embeddings) with lightweight reranking applied after candidate generation achieves sufficient recall, rather than applying a generic cross-encoder blindly.

5. AI Tools Statement

AI tools (ChatGPT/Codex, Claude) were used as engineering assistants: debugging notebook code, adapting the SciFact tutorial to the RapidFire documentation corpus, writing utility scripts (`generate_and_judge_configs.py`, `extract_all_config_results.py`), summarizing experiment outputs, drafting report language, identifying likely failure modes, and brainstorming evaluation questions. All generated code and report content was reviewed and validated by the team. The pipeline was executed locally; output JSON files were schema-checked; context lengths were verified against the 2,000-token budget; and results were collected from RapidFire MLflow logs. Design decisions were based on observed metrics and manual output inspection, not accepted uncritically from AI suggestions. Golden Q&A pairs were grounded in source documentation with explicit file and line evidence and checked for corpus-answerability before inclusion.